

CNN-Based Automated Classification of SEM Microstructures

使用 InceptionV3 與 Transfer Learning 實作 10 類微結構識別

作者：梁家毓 E54111213

國立成功大學 材料科學與工程學系



Introduction

專案動機

掃描式電子顯微鏡 (SEM) 產生大量影像，人工分類耗時且仰賴專家經驗，限制了研究效率與一致性。

專案目標

建立一套自動化深度學習模型，能高效率且精準地識別 10 種不同的材料微結構，加速科學發現。

應用場景

- 材料表徵自動化 (Automated Characterization)
- 製程品質管制 (QA/QC)，即時監控生產線
- 大型數據庫的自動標籤與索引整理

Dataset Overview

類別列表 (10 Classes)

- 資料來源：Kaggle 與 EUDAT，確保資料來源的豐富性與多樣性。
- 資料規模：涵蓋 10 個獨特類別的 SEM 影像，每類包含數百至數千張圖片。

1 Biological (生物)

2 Fibres (纖維)

3 Films_Coated_Surface (薄膜/塗層)

4 MEMS_devices (微機電元件)

5 Nanowires (奈米線)

6 Particles (顆粒)

7 Patterned_surface (圖案化表面)

8 Porous_Sponge (多孔/海綿狀)

9 Powder (粉末)

10 Tips (探針/尖端)



資料預處理流程 (Data Preprocessing Pipeline)



資料清洗

過濾非影像檔案，確保副檔名正確，避免模型讀取到無效數據。



資料集切割 (Data Splitting)

採用 Stratified Split (分層抽樣) 確保各類別比例在訓練、驗證、測試集中一致，避免數據偏差。



比例配置

- Training set: 70%
- Validation set: 15%
- Test set: 15%



實體隔離

程式碼自動建立 train, val, test 資料夾並複製檔案，徹底避免 Data Leakage，確保測試結果的可靠性。

資料增強策略 (Data Augmentation)

像素調整

將像素值從 $[0, 255]$ 縮放到 $[0, 1]$ ，加速模型收斂並提升性能。

輕微旋轉

模擬 SEM 樣品擺放角度的細微差異，讓模型對角度變化更具魯棒性。

視野平移

模擬採樣過程中視野的輕微偏移，增加模型對物件位置變化的容忍度。

放大倍率差異

模擬 SEM 不同的放大倍率，提升模型對不同尺度特徵的辨識能力。

水平翻轉

利用圖像的對稱性增加數據量，幫助模型學習不依賴方向的特徵

輸入尺寸：所有影像均被調整為 299 x 299 像素，以完美配合 InceptionV3 模型所需的標準輸入尺寸

SEM 影像中，結構的方向和大小可能會因為拍攝條件而異。因此，我在訓練過程中引入了即時的資料增強，包括旋轉、平移和縮放，讓模型學會辨認結構的本質，而不是死記硬背圖片的位置。

```
data_root = "/content/sem_dataset_splits"
img_size = (299, 299) # InceptionV3 預設 299x299
batch_size = 32

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    width_shift_range=0.05,
    height_shift_range=0.05,
    zoom_range=0.1,
    horizontal_flip=True,
)
```

模型架構設計 (Model Architecture)

整體架構設計

兩個主要部分：「強大的預訓練骨幹」+「我自定義的分類頭」

特徵提取器

分類器

Backbone (骨幹)：InceptionV3

- 預訓練：於 ImageNet 資料集上進行預訓練。
- 選擇原因：Inception 架構擅長捕捉多尺度的特徵，特別適合處理紋理複雜且多樣的 SEM 圖像。

Custom Head (分類器)

GlobalAveragePooling2D

將骨幹輸出的特徵圖降維，防止 overfitting。

Dropout(0.3)

以 30% 的機率隨機關閉神經元，有效防止模型過擬合訓練數據。

Dense(256, activation='relu')

全連接層，具有 256 個神經元和 ReLU 激活函數，用於特徵整合與非線性轉換。

Dense(10, activation='softmax')

最終輸出層，10 個神經元對應 10 個類別，使用 Softmax 激活函數輸出各類別機率

```
from tensorflow.keras import layers, models
from tensorflow.keras.applications import InceptionV3

num_classes = train_gen.num_classes # 應該是 10
input_shape = (299, 299, 3)

base_model = InceptionV3(
    weights='imagenet',
    include_top=False,
    input_shape=input_shape
)

# 先凍結 backbone
base_model.trainable = False

x = base_model.output
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(num_classes, activation='softmax')(x)

model = models.Model(inputs=base_model.input, outputs=outputs)

model.summary()
```

訓練結果

訓練過程中使用了多種 Callback 機制來監控訓練狀況
特別是動態調整學習率 (ReduceLR) 和提早停止 (EarlyStopping)
這能確保模型收斂在最佳狀態，避免訓練過度。

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its constructor.  
self._warn_if_super_not_called()  
Epoch 1/5  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 9s/step - accuracy: 0.3712 - loss: 1.8891  
Epoch 1: val_accuracy improved from -inf to 0.72812, saving model to best_sem_cnn.h5  
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format via `model.save(format='keras')` or `keras.saving.save_model(model, save_format='keras')`.  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 637s 13s/step - accuracy: 0.3739 - loss: 1.8808 - val_accuracy: 0.7281 - val_loss: 0.8615 - learning_rate: 0.0010  
Epoch 2/5  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 9s/step - accuracy: 0.6701 - loss: 0.9735  
Epoch 2: val_accuracy improved from 0.72812 to 0.76562, saving model to best_sem_cnn.h5  
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format via `model.save(format='keras')` or `keras.saving.save_model(model, save_format='keras')`.  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 617s 12s/step - accuracy: 0.6705 - loss: 0.9724 - val_accuracy: 0.7656 - val_loss: 0.7262 - learning_rate: 0.0010  
Epoch 3/5  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 9s/step - accuracy: 0.7460 - loss: 0.8078  
Epoch 3: val_accuracy improved from 0.76562 to 0.78438, saving model to best_sem_cnn.h5  
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format via `model.save(format='keras')` or `keras.saving.save_model(model, save_format='keras')`.  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 606s 12s/step - accuracy: 0.7461 - loss: 0.8066 - val_accuracy: 0.7844 - val_loss: 0.6655 - learning_rate: 0.0010  
Epoch 4/5  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 9s/step - accuracy: 0.7382 - loss: 0.7600  
Epoch 4: val_accuracy did not improve from 0.78438  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 620s 13s/step - accuracy: 0.7385 - loss: 0.7595 - val_accuracy: 0.7812 - val_loss: 0.6209 - learning_rate: 0.0010  
Epoch 5/5  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 9s/step - accuracy: 0.7869 - loss: 0.6558  
Epoch 5: val_accuracy improved from 0.78438 to 0.80156, saving model to best_sem_cnn.h5  
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format via `model.save(format='keras')` or `keras.saving.save_model(model, save_format='keras')`.  
50/50 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 624s 13s/step - accuracy: 0.7869 - loss: 0.6559 - val_accuracy: 0.8016 - val_loss: 0.5946 - learning_rate: 0.0010  
Restoring model weights from the end of the best epoch: 5.
```

模型評估結果 (Evaluation Results)

混淆矩陣 (Confusion Matrix)

展示模型對每個類別的預測情況，對角線為正確預測。

Precision (精確率)

模型預測為正例中，真正為正例的比例。

Recall (召回率)

所有真正為正例中，模型成功預測為正例的比例

F1-Score

Precision 和 Recall 的調和平均數，綜合衡量模型性能

這是模型在從未見過的測試集上的表現

Confusion matrix (raw counts):

[[129	0	0	1	9	2	0	0	1	1]
[1	9	0	6	6	0	1	0	0	0]
[2	0	4	0	20	17	0	0	3	0]
[4	1	0	554	27	2	28	0	0	8]
[8	3	0	8	488	13	5	0	16	7]
[4	0	0	5	23	462	2	0	11	5]
[0	0	0	110	18	16	346	0	0	7]
[7	0	0	2	2	3	2	7	3	0]
[6	0	1	0	14	6	1	0	105	1]
[2	0	0	27	14	1	3	0	0	187]]

```
test_loss, test_acc = model.evaluate(test_gen)
print(f"Test loss = {test_loss:.4f}, Test accuracy = {test_acc:.4f}")
```

88/88 728s 8s/step - accuracy: 0.8336 - loss: 0.5121
Test loss = 0.5455, Test accuracy = 0.8220

實際演示

影像預測

- 輸入：單張未經標註的 SEM 圖片。
- 輸出：模型會即時返回預測的類別標籤與對應的信心分數（Confidence Score）。

Batch shape: (32, 299, 299, 3)
1/1 3s 3s/step
True label: Biological
Predicted label: Biological
Class probabilities:
Biological : 0.9594
Fibres : 0.0011
Films_Coated_Surface : 0.0006
MEMS_devices_and_electrodes : 0.0014
Nanowires : 0.0332
Particles : 0.0003
Patterned_surface : 0.0003
Porous_Sponge : 0.0007
Powder : 0.0011
Tips : 0.0020

True: Biological
Pred: Biological

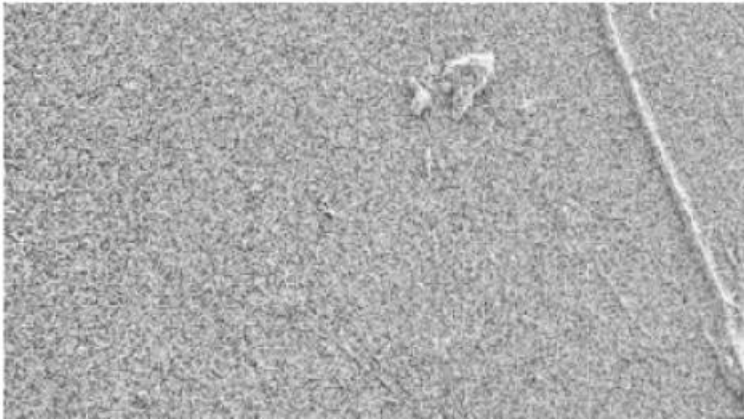


Image path: /content/sample_data/sem_sample/NiCo-5.png
Predicted label: Particles
Class probabilities:
Biological : 0.0166
Fibres : 0.0036
Films_Coated_Surface : 0.0020
MEMS_devices_and_electrodes : 0.0259
Nanowires : 0.0473
Particles : 0.8122
Patterned_surface : 0.0231
Porous_Sponge : 0.0186
Powder : 0.0129
Tips : 0.0377

Pred: Particles

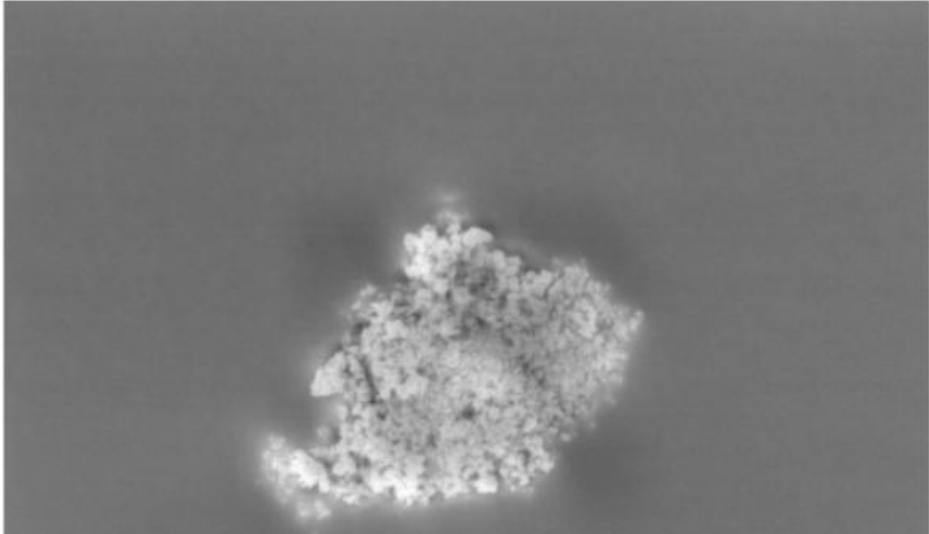
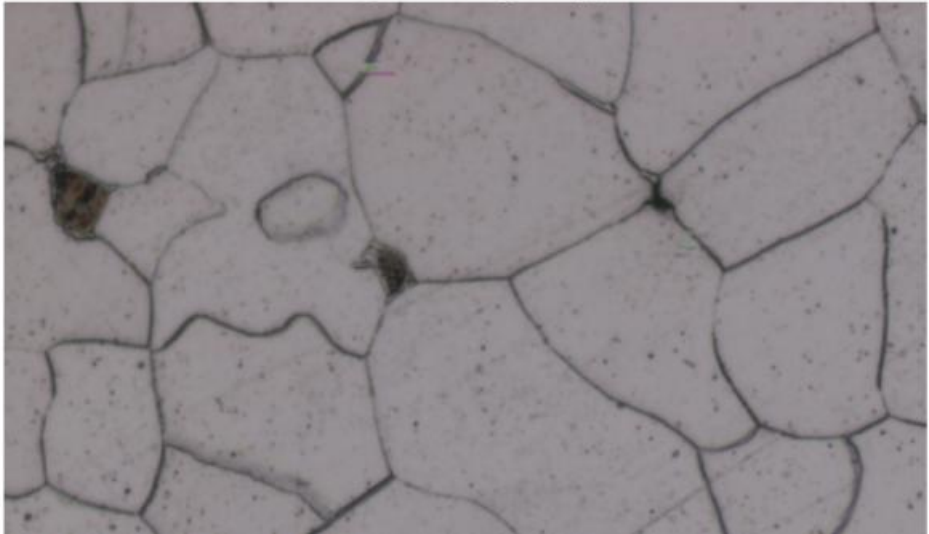


Image path: /content/sample_data/100-100-6.5-500x.jpg
Predicted label: MEMS_devices_and_electrodes
Class probabilities:
Biological : 0.0085
Fibres : 0.0051
Films_Coated_Surface : 0.1939
MEMS_devices_and_electrodes : 0.2864
Nanowires : 0.0131
Particles : 0.1641
Patterned_surface : 0.2072
Porous_Sponge : 0.0420
Powder : 0.0710
Tips : 0.0087

Pred: MEMS_devices_and_electrodes



參考文獻：

Dataset: <https://www.kaggle.com/datasets/sharumaan/semimages-299x299>

[1] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.

[2] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.